

Portafolio Personal

Documentación técnica del proyecto

RESUMEN

Portafolio Personal es un sitio interactivo que muestra proyectos de ciencia e ingeniería de datos. Está construido con React, TypeScript y Vite, y cuenta con animaciones 3D y un chat asistente.

Características principales

- Hash routing custom
- Carpetas 3D animadas
- Lightbox con navegación circular
- Grafo conceptual force-directed
- CV interactivo
- Blog self-hosted bilingüe

STACK TECNOLÓGICO

Core

React 18 sobre TypeScript 5.6 con modo estricto, empaquetado con Vite 5.4.

Estilos y animaciones

Tailwind CSS, lottie-react para la animación del hero, CSS 3D transforms para las carpetas, simulación force-directed propia en SVG para el grafo.

APIs del navegador

Web Audio API con GainNode, localStorage, Pointer Events para drag y pinch zoom, Page Visibility para pausar música, MediaQuery para detectar hover y preferencia de tema.

Infraestructura

pnpm como gestor obligatorio, GitHub Actions para CI, GitHub Pages para hosting estático global con HTTPS.

Bundle final

Aproximadamente 180 kilobytes comprimidos. Sin React Router, Framer Motion ni librerías de state management.

ARQUITECTURA DEL FRONTEND

La aplicación se estructura en cinco capas verticales que separan responsabilidades: herramientas de desarrollo, framework principal, estilos e interfaz, APIs del navegador y despliegue continuo. Cada capa solo depende de las superiores y expone una interfaz acotada a las inferiores.

Capas verticales

Dev/Build (pnpm, TypeScript, Vite) que solo dependen de las herramientas del sistema. Core (React, Hash Routing) que monta el sitio. Estilos y UI (Tailwind, lucide, lottie, CSS 3D) que dan la identidad visual. Browser APIs (Web Audio, localStorage, Pointer Events, Page Visibility, MediaQuery) que aprovechan capacidades nativas sin libs adicionales. CI/Deploy (GitHub Actions y Pages) que lleva el bundle compilado a producción.

Decisión clave

Hash routing custom de aproximadamente treinta líneas reemplaza a react-router, lo que elimina una dependencia significativa y mantiene URLs compartibles que sobreviven a refrescos del navegador. Ver el diagrama completo del stack en la sección Diagramas del portafolio.

COMPONENTES PRINCIPALES

FolderPortfolio

Renderiza el grid de seis carpetas tridimensionales con abanico de cards y orquesta el lightbox modal.

ProjectDetailView

Vista de detalle con hero, grafo conceptual force-directed y seis tarjetas de sección. Navegación entre proyectos sin volver al grid.

ConceptGraph

Simulación física propia con repulsión entre nodos, springs en aristas y levitación continua. Soporta drag con pointer events y filtro por grupo.

DiagramsViewer

Visor fullscreen con zoom hasta ochocientos por ciento, pan y pinch zoom en mobile. Navegación circular entre múltiples imágenes.

PortfolioChat

Burbuja flotante con panel desplegable. Mock por keyword matching, integración con LLM planeada como próxima fase.

ARQUITECTURA DEL BACKEND

Planificación de una capa serverless mínima que coexiste con el frontend estático sin convertirlo en monolito. Dos workers en runtime para chat y administración, una suite de agentes batch en GitHub Actions para auto-generar contenido del portafolio.

Tres capas funcionales

Eventos (push a repo, release tag, cron, chat de visitante, aprobación de admin) que disparan la cadena. Backend (workers de runtime para chat y admin de borradores, agentes batch en GitHub Actions agrupados en content, lifecycle y distribution, persistencia mixta entre JSON committed y Workers KV ephemeral). Outputs (portafolio web auto-desplegado, CV PDF auto-generado, blog en Dev.to, post en LinkedIn, respuesta del chat).

Decisión clave

JAMstack híbrida en lugar de monolito Express o n8n. El tráfico impredecible de un portafolio personal justifica pagar por request en Cloudflare Workers en lugar de mantener un servidor 24/7. Ver el diagrama completo del flujo en la sección Diagramas del portafolio.

MODELO DE DATOS

El catálogo de proyectos vive en `src/data/projects.ts` como un arreglo de seis categorías. Cada proyecto declara identificador único, imagen, título y descripción bilingües, lista de tags, estado dentro de tres valores, enlaces a repositorio y demo, conceptos del grafo y categorías cross-disciplina opcionales.

Tipos clave

LocalizedString, ProjectStatus, ConceptGroup con cuatro valores semánticos: arquitectura, datos, operaciones y machine learning.

Persistencia

JSON y markdown committed al repositorio para datos públicos. Workers KV para rate limiting del chat y cola de borradores de distribución. No se almacena historial de conversaciones por privacidad.

PRÓXIMOS PASOS Y CAMBIOS RECIENTES

Cambios recientes

Se han agregado recientemente soporte para repos externos en agentes, se ha actualizado la documentación y se ha mejorado la funcionalidad del chat asistente. También se ha agregado un blog self-hosted bilingüe.

Próximos pasos

- Implementar nuevos proyectos
- Mejorar la experiencia del usuario
- Agregar funcionalidades al chat asistente
- Optimizar el rendimiento del sitio